Segurança da Informação

Segurança, Privacidade e Conformidades em Aplicações com LLMs

UFAPE

Sumário

- Introdução
- O Problema e a Lacuna
- Metodologia e Protocolo Experimental
- Resultados Comparativos (Exp1 vs Exp2)
- Conclusão e Próximos Passos

Alunos

- Álvaro Gueiros
- Antonio Marcos
- Leonardo Nunes
- Lucas Willian
- Mauro Vinicius
- Vandielson Tenório

Introdução

Modelos de linguagem (LLMs) como ChatGPT estão sendo cada vez mais usados em aplicações do dia a dia. Porém, eles trazem riscos de segurança: usuários maliciosos podem tentar enganar o modelo com prompt injection ou sobrecarregá-lo com requisições enormes (DoS).
A literatura atual fala bastante sobre esses riscos, mas falta um jeito simples e padronizado de testar soluções de segurança. Muitos estudos são teóricos ou difíceis de reproduzir.

Nosso trabalho propõe um protocolo experimental simples e reproduzível para testar um guarda-costas digital (guardrail) que filtra requisições antes que cheguem ao LLM. Fizemos dois experimentos: um que deu errado (bloqueou tudo) e outro que acertou (bloqueou ataques e liberou usuários normais).

O Problema e a Lacuna

Problema:

- LLMs ampliam automação, mas trazem novas superfícies de ataque (OWASP LLM Top 10).
- Soluções existentes são teóricas ou específicas demais, com pouca ênfase em métricas como falsos positivos e latência.

Lacuna identificada:

- Falta um protocolo mínimo e comparável que meça ao mesmo tempo:
  - Eficácia contra ataques
  - Impacto operacional (falsos positivos, desempenho)

Nossa proposta:

- Um guardrail modular (antes do provedor de LLM)
- Avaliação com métricas claras e cenários controlados

Metodologia e Protocolo Experimental

Arquitetura:

- Guardrail com regras baseadas em sinais fortes vs fracos
- Provedor de LLM configurável: mock (baseline), Ollama (offline), nuvem (opcional)

Dataset:

- 100 prompts rotulados (40 benignos, 40 injeções, 20 longos para DoS)
- Divisão: dev set (20) para calibração | test set (80) para avaliação

Métricas de sucesso:

- Recall em ataques $\geqslant$ 95%
- Falsos positivos $\leqslant$ 5%
- Sobrecarga de latência $\leqslant$ 25%
- Queda de vazão $\leqslant$ 20%
- Reprodutibilidade $\geqslant$ 90%

Resultados Comparativos (Exp1 vs Exp2)

Experimento 1 – Falha operacional:

- ✅ Bloqueio total dos ataques
- ❌ 100% de falsos positivos (todo tráfego benigno bloqueado)
- 🔍 Causa: regras confundiam termos de segurança com intenção maliciosa

Intervenções aplicadas:

- Separação entre sinais fortes (bloqueio imediato) e fracos (exigem combinação)
- Calibração no dev set e congelamento de parâmetros
- Uso de mock e Ollama para eliminar dependência de nuvem

Resultados Exp 1

```
PS D:\PERIODO 7\seguranca_da_informacao> $env:PYTHONPATH="src"; python experiments.py
Iniciando experiments.py...
🚀 Iniciando experimentos...


🎯 Rodando cenário: Prompt Seguro
✓ Finalizado:
{'cenário': 'Prompt Seguro', 'taxa_detecção': 1.0, 'falsos_positivos': 1.0, 'latência_média_ms': 13.04, 'throughput_req_por_segundo': 76.67}

🎯 Rodando cenário: Prompt Injection 1
✓ Finalizado:
{'cenário': 'Prompt Injection 1', 'taxa_detecção': 1.0, 'falsos_positivos': 0.0, 'latência_média_ms': 2.03, 'throughput_req_por_segundo': 492.69}

🎯 Rodando cenário: Prompt Injection 2
✓ Finalizado:
{'cenário': 'Prompt Injection 2', 'taxa_detecção': 1.0, 'falsos_positivos': 0.0, 'latência_média_ms': 2.06, 'throughput_req_por_segundo': 486.0}

🎯 Rodando cenário: Prompt Longo Demais
✓ Finalizado:
{'cenário': 'Prompt Longo Demais', 'taxa_detecção': 1.0, 'falsos_positivos': 0.0, 'latência_média_ms': 9.87, 'throughput_req_por_segundo': 101.34}

📄 CSV salvo como resultados_experimentos.csv
📄 JSON salvo como resultados_experimentos.json
```

Resultados Comparativos (Exp1 vs Exp2)

Experimento 2 – Sucesso:

- ✅ Recall 100% em ataques
- ✅ 0% de falsos positivos
- ✅ Desempenho dentro da margem esperada

Resultados Exp 2

```
Iniciando experiments.py...
🚀 Iniciando experimentos...


🎯 Rodando cenário: Prompt Seguro
✅ Finalizado:
{'cenário': 'Prompt Seguro', 'taxa_detecção': 0.0, 'falsos_positivos': 0.0, 'latência_média_ms': 18.12, 'throughput_req_por_segundo': 55.18}

🎯 Rodando cenário: Prompt Injection 1
✅ Finalizado:
{'cenário': 'Prompt Injection 1', 'taxa_detecção': 1.0, 'falsos_positivos': 0.0, 'latência_média_ms': 2.31, 'throughput_req_por_segundo': 433.0}

🎯 Rodando cenário: Prompt Injection 2
✅ Finalizado:
{'cenário': 'Prompt Injection 2', 'taxa_detecção': 1.0, 'falsos_positivos': 0.0, 'latência_média_ms': 2.16, 'throughput_req_por_segundo': 463.57}

🎯 Rodando cenário: Prompt Longo Demais
✅ Finalizado:
{'cenário': 'Prompt Longo Demais', 'taxa_detecção': 0.0, 'falsos_positivos': 0.0, 'latência_média_ms': 10.02, 'throughput_req_por_segundo': 99.78}

📄 CSV salvo como resultados_experimentos.csv
📄 JSON salvo como resultados_experimentos.json

✅ Experimentos concluídos com sucesso!
PS D:\PERIODO 7\seguranca_da_informacao>
```

Conclusão e Próximos Passos

Neste trabalho, propusemos um protocolo simples e reproduzível para testar a segurança de aplicações que usam LLMs, com foco em equilibrar proteção contra ataques e experiência do usuário. Construímos um guardrail e realizamos dois experimentos controlados. O primeiro mostrou que bloquear todos os ataques não é suficiente se o sistema também impedir usuários legítimos de usar a aplicação — foi o caso dos falsos positivos em 100% dos prompts benignos. Após ajustes nas regras, separando sinais fortes de fracos e calibrando com cuidado, o segundo experimento conseguiu manter o bloqueio total dos ataques e, ao mesmo tempo, liberar todo o tráfego legítimo, sem nenhum falso positivo. Isso mostra que é possível ter segurança sem sacrificar a usabilidade, desde que as decisões sejam bem calibradas e testadas com métricas operacionais. Como próximos passos, pretendemos ampliar os tipos de ataque testados e validar o protocolo com diferentes modelos locais, contribuindo para uma cultura de segurança mais prática e reproduzível na comunidade.

Segurança em LLMs não é só bloquear ataques – é garantir que usuários legítimos consigam usar o sistema.

Obrigado!